

程式設計 Computational Thinking and Programming

電腦裡面到底在幹嘛？

張賢宗

2-1 INTRODUCTION

A **number system** defines how a number can be represented using distinct symbols. A number can be represented differently in different systems. For example, the two numbers $(2A)_{16}$ and $(52)_8$ both refer to the same quantity, $(42)_{10}$, but their representations are different. Several number systems have been used in the past and can be categorized into two groups: **positional** and **non-positional** systems. Our main goal is to discuss the positional number systems, but we also give examples of non-positional systems.



2-2 POSITIONAL NUMBER SYSTEMS

In a **positional number system**, the position a symbol occupies in the number determines the value it represents. In this system, a number represented as:

$$\pm (S_{k-1} \dots S_2 S_1 S_0 \cdot S_{-1} S_{-2} \dots S_{-l})_b$$

has the value of:

$$n = \pm [S_{k-1} \times b^{k-1} + \dots + S_1 \times b^1 + S_0 \times b^0] + [S_{-1} \times b^{-1} + S_{-2} \times b^{-2} + \dots + S_{-l} \times b^{-l}]$$

in which S is the set of symbols, b is the **base** (or **radix**).

The decimal system (base 10)

The word decimal is derived from the Latin root **decem** (ten). In this system the **base $b = 10$** and we use ten symbols

$$S = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

The symbols in this system are often referred to as **decimal digits** or just **digits**.

Integers

$$N = \pm S_{k-1} \times 10^{k-1} + S_{k-2} \times 10^{k-2} + \dots + S_2 \times 10^2 + S_1 \times 10^1 + S_0 \times 10^0$$

	10^{k-1}	10^{k-2}	$\dots$	10^2	10^1	10^0	Place values
$\pm$	S_{k-1}	S_{k-2}	$\dots$	S_2	S_1	S_0	Number
	↓	↓		↓	↓	↓	
$N = \pm$	$S_{k-1} \times 10^{k-1}$	$+ S_{k-2} \times 10^{k-2}$	$+ \dots$	$+ S_2 \times 10^2$	$+ S_1 \times 10^1$	$+ S_0 \times 10^0$	Values

Figure 2.1 Place values for an integer in the decimal system

Example 2.1

The following shows the place values for the integer +224 in the decimal system.

		10^2		10^1		10^0	Place values
		2		2		4	Number
N	$=$	$+$	2×10^2	$+$	2×10^1	$+$	4×10^0
							Values

Note that the digit 2 in position 1 has the value 20, but the same digit in position 2 has the value 200. Also note that we normally drop the plus sign, but it is implicit.

Example 2.2

The following shows the place values for the decimal number -7508 . We have used 1, 10, 100, and 1000 instead of powers of 10.

	1000	100	10	1	Place values
	7	5	0	8	Number
N	$=$	$-$	$($	7×1000 $+$ 5×100 $+$ 0×10 $+$ 8×1	$)$ Values

Reals

Integral part

Fractional part

$$R = \pm \left(S_{k-1} \times 10^{k-1} + \dots + S_1 \times 10^1 + S_0 \times 10^0 \right) + \left(S_{-1} \times 10^{-1} + \dots + S_{-l} \times 10^{-l} \right)$$

Example 2.3

The following shows the place values for the real number +24.13.

	10^1	10^0		10^{-1}	10^{-2}	Place values
	2	4	•	1	3	Number
$R = +$	2×10	$+ 4 \times 1$	$+$	1×0.1	$+ 3 \times 0.01$	Values

The binary system (base 2)

The word binary is derived from the Latin root **bini** (or two by two). In this system the **base b = 2** and we use only two symbols,

$$S = \{0, 1\}$$

The symbols in this system are often referred to as **binary digits** or **bits** (binary digit).

Integers

$$N = \pm S_{k-1} \times 2^{k-1} + S_{k-2} \times 2^{k-2} + \dots + S_2 \times 2^2 + S_1 \times 2^1 + S_0 \times 2^0$$

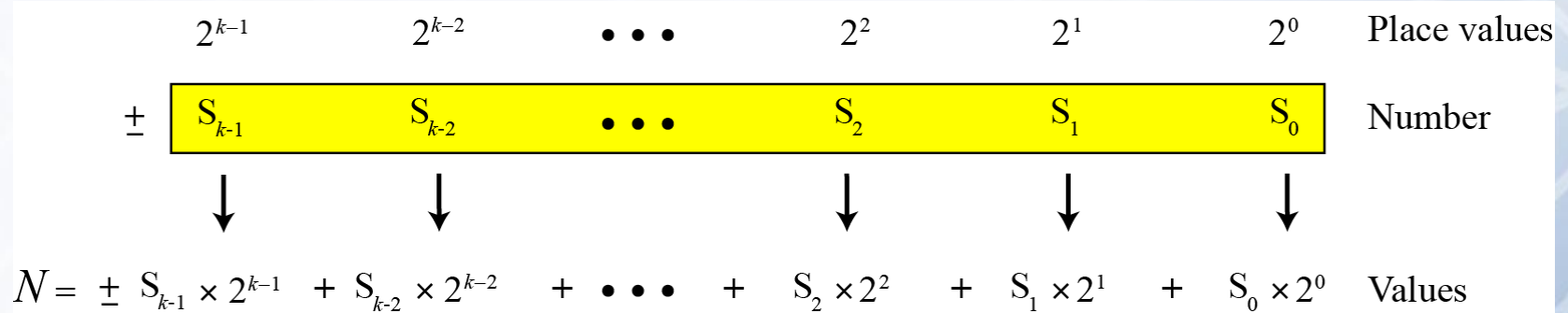


Figure 2.2 Place values for an integer in the binary system

Example 2.4

The following shows that the number $(11001)_2$ in binary is the same as 25 in decimal. The subscript 2 shows that the base is 2.

	2^4	2^3	2^2	2^1	2^0	Place values				
	1	1	0	0	1	Number				
$N =$	1×2^4	$+$	1×2^3	$+$	0×2^2	$+$	0×2^1	$+$	1×2^0	Decimal

The equivalent decimal number is $N = 16 + 8 + 0 + 0 + 1 = 25$.


$$R = \pm \left(\begin{array}{c} \text{Integral part} \\ s_{k-1} \times 2^{k-1} + \dots + s_1 \times 2^1 + s_0 \times 2^0 \end{array} \right) + \left(\begin{array}{c} \text{Fractional part} \\ s_{-1} \times 2^{-1} + \dots + s_{-l} \times 2^{-l} \end{array} \right)$$

The following shows that the number $(101.11)_2$ in binary is equal to the number 5.75 in decimal.

2.12



widelab
Web Information and Data Engineering Laboratory

The hexadecimal system (base 16)

The word **hexadecimal** is derived from the Greek root **hex** (six) and the Latin root **decem** (ten). In this system the **base $b = 16$** and we use sixteen symbols to represent a number. The set of symbols is

$$S = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$$

Note that the symbols A, B, C, D, E, F are equivalent to 10, 11, 12, 13, 14, and 15 respectively. The symbols in this system are often referred to as **hexadecimal digits**.

Integers

$$N = \pm S_{k-1} \times 16^{k-1} + S_{k-2} \times 16^{k-2} + \dots + S_2 \times 16^2 + S_1 \times 16^1 + S_0 \times 16^0$$

	16^{k-1}	16^{k-2}	$\dots$	16^2	16^1	16^0	Place values
$\pm$	S_{k-1}	S_{k-2}	$\dots$	S_2	S_1	S_0	Number
	↓	↓		↓	↓	↓	
$N =$	$\pm S_{k-1} \times 16^{k-1}$	$+ S_{k-2} \times 16^{k-2}$	$+ \dots$	$+ S_2 \times 16^2$	$+ S_1 \times 16^1$	$+ S_0 \times 16^0$	Values

Figure 2.3 Place values for an integer in the hexadecimal system

Example 2.6

The following shows that the number $(2AE)_{16}$ in hexadecimal is equivalent to 686 in decimal.

	16^2	16^1	16^0	Place values
	2	A	E	Number
N =	2×16^2	+ 10×16^1	+ 14×16^0	Values

The equivalent decimal number is $N = 512 + 160 + 14 = 686$.

The octal system (base 8)

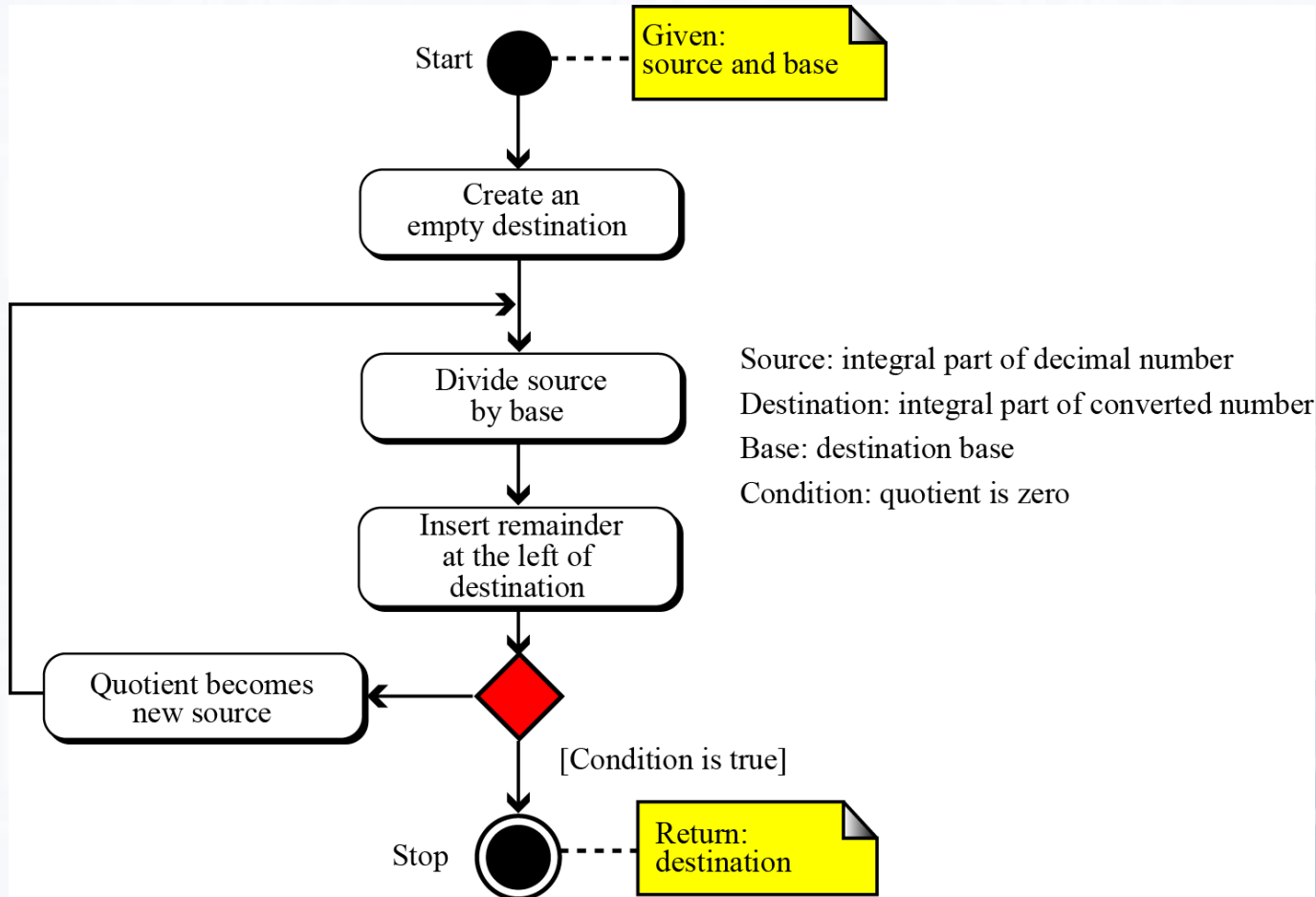
The word octal is derived from the Latin root **octo** (eight). In this system the **base $b = 8$** and we use eight symbols to represent a number. The set of symbols is

$$S = \{0, 1, 2, 3, 4, 5, 6, 7\}$$

Kahoot 一下

❖ <https://play.kahoot.it/v2/?quizId=ebc14b37-e790-48c0-875e-a039eac10c5d>

Decimal to any base



2.18 Figure 2.6 Converting other bases to decimal (integral part)

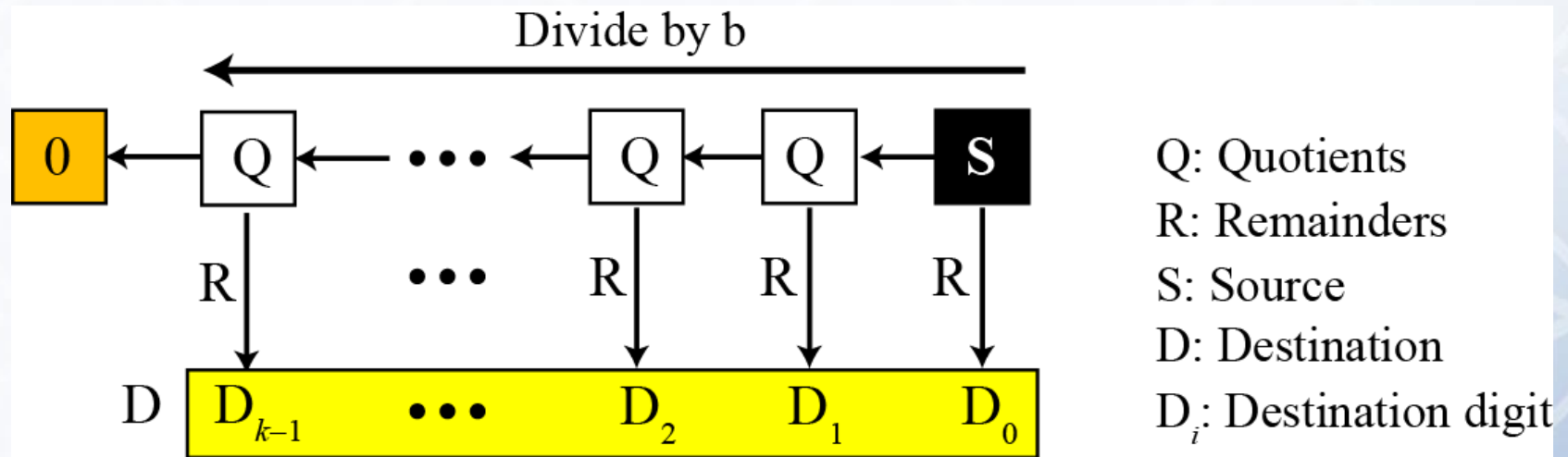
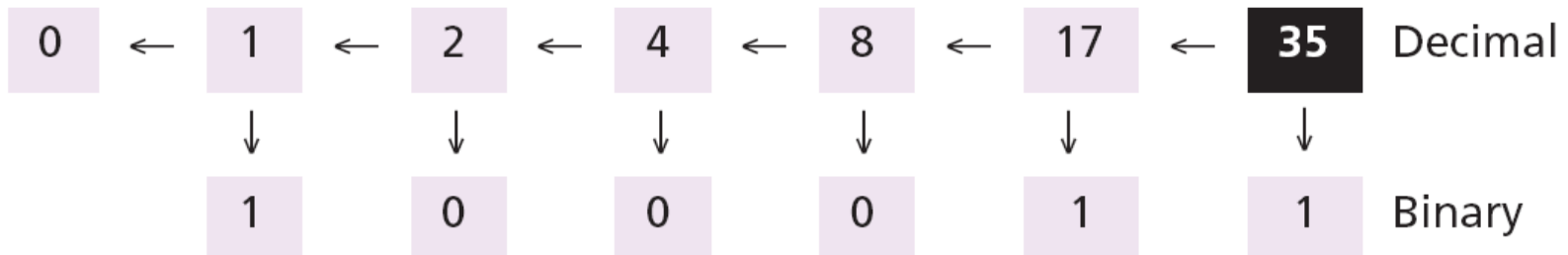
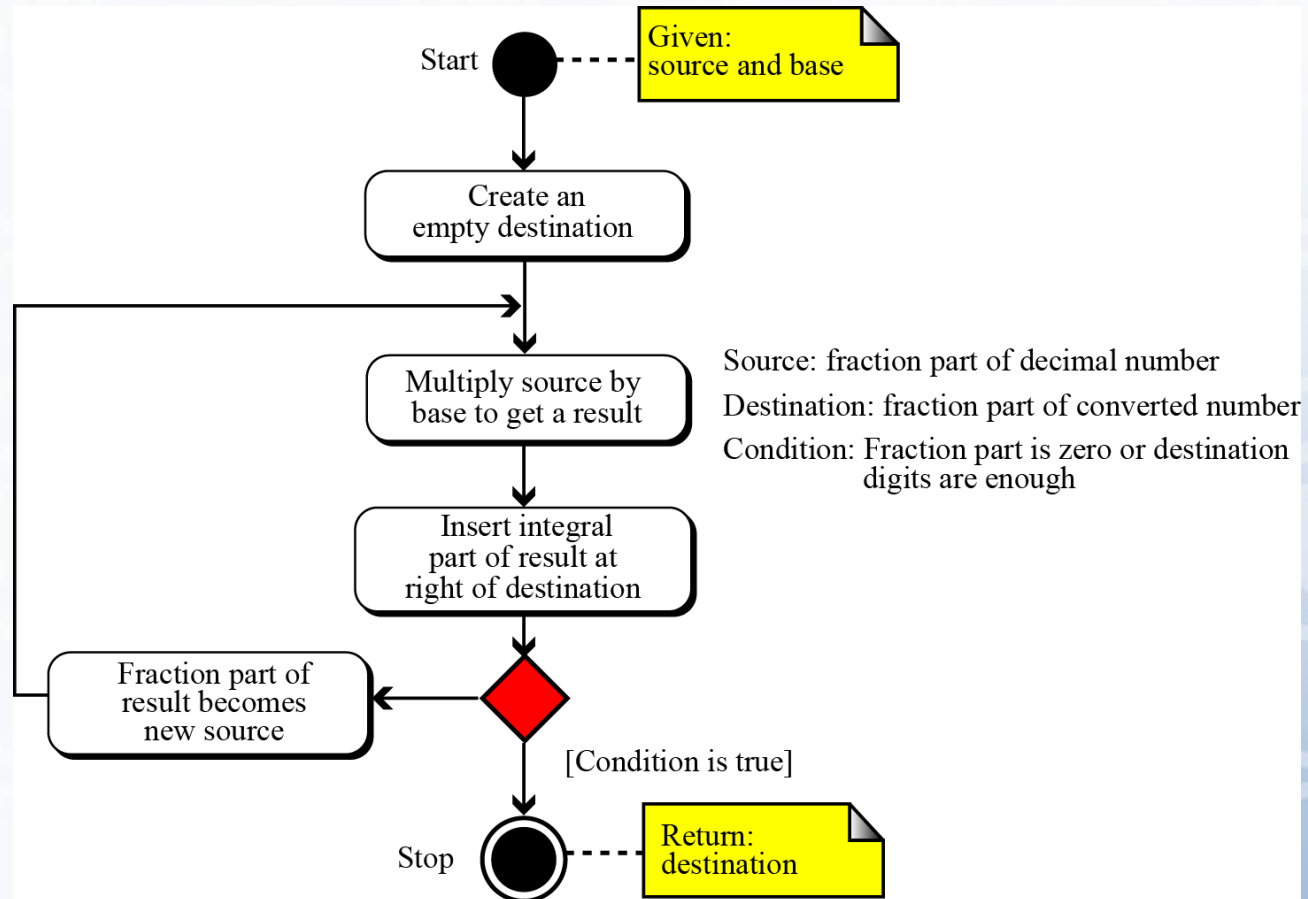


Figure 2.7 Converting the integral part of a number in decimal to other bases

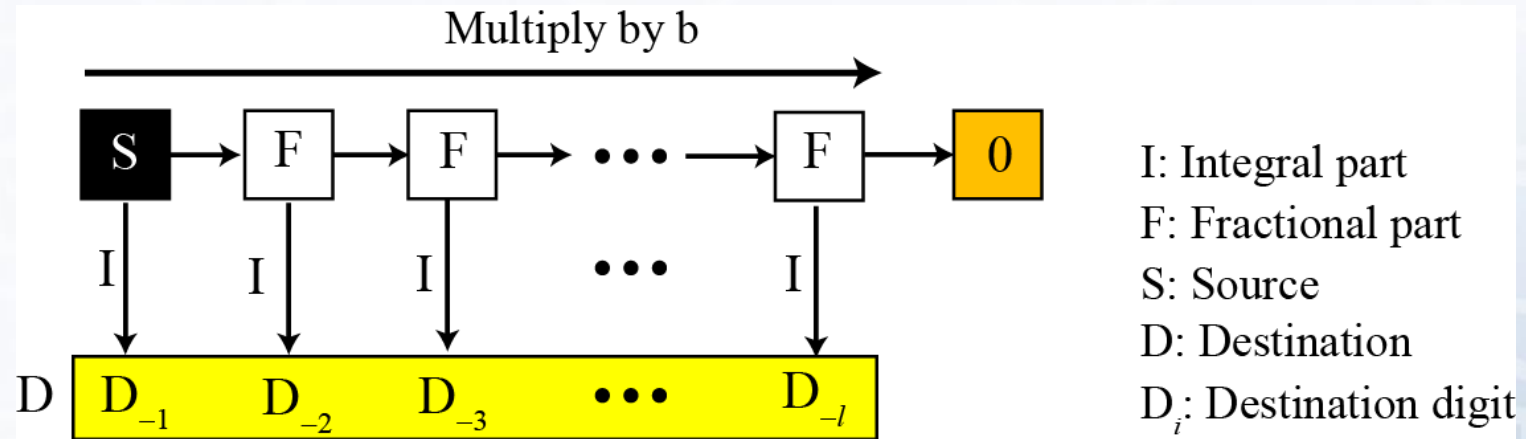
Example 2.11

The following shows how to convert 35 in decimal to binary. We start with the number in decimal, we move to the left while continuously finding the quotients and the remainder of division by 2. The result is $35 = (100011)_2$.





2.21 **Figure 2.8** Converting the fractional part of a number in decimal to other bases



Note:

The fraction may never become zero.

Stop when enough digits have been created.

Figure 2.9 Converting the fractional part of a number in decimal to other bases

Example 2.14

Convert the decimal number 0.625 to binary.

Decimal	0.625	→	0.25	→	0.50	→	0.00
	↓		↓		↓		
Binary	• 1		0		1		

Since the number $0.625 = (0.101)_2$ has no integral part, the example shows how the fractional part is calculated.

Binary-hexadecimal conversion

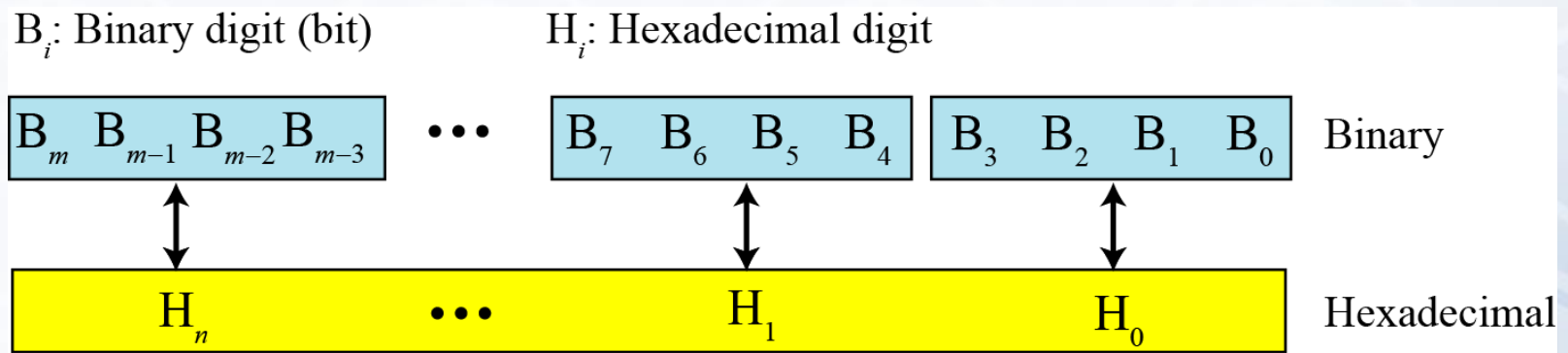


Figure 2.10 Binary to hexadecimal and hexadecimal to binary conversion

Example 2.19

Show the hexadecimal equivalent of the binary number $(110011100010)_2$.

Solution

We first arrange the binary number in 4-bit patterns:

100 1110 0010

Note that the leftmost pattern can have one to four bits. We then use the equivalent of each pattern shown in Table 2.2 on page 25 to change the number to hexadecimal: $(4E2)_{16}$.

Table 2.2 Comparison of numbers in the four systems

Decimal	Binary	Octal	Hexadecimal
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Example 2.20

What is the binary equivalent of $(24C)_{16}$?

Solution

Each hexadecimal digit is converted to 4-bit patterns:

$2 \rightarrow 0010$, $4 \rightarrow 0100$, and $C \rightarrow 1100$

The result is $(001001001100)_2$.

2-3 NONPOSITIONAL NUMBER SYSTEMS

Although **non-positional number systems** are not used in computers, we give a short review here for comparison with positional number systems. A non-positional number system still uses a limited number of symbols in which each symbol has a value. However, the position a symbol occupies in the number normally bears no relation to its value—the value of each symbol is fixed. To find the value of a number, we add the value of all symbols present in the representation.

Example 2.24

Roman numerals are a good example of a non-positional number system. This number system has a set of symbols $S = \{I, V, X, L, C, D, M\}$. The values of each symbol are shown in Table 2.3

Table 2.3 Values of symbols in the Roman number system

<i>Symbol</i>	<i>I</i>	<i>V</i>	<i>X</i>	<i>L</i>	<i>C</i>	<i>D</i>	<i>M</i>
Value	1	5	10	50	100	500	1000

To find the value of a number, we need to add the value of symbols subject to specific rules (See the textbook).

Example 2.24 (Continued)

The following shows some Roman numbers and their values.

III	→	$1 + 1 + 1$	=	3
IV	→	$5 - 1$	=	4
VIII	→	$5 + 1 + 1 + 1$	=	8
XVIII	→	$10 + 5 + 1 + 1 + 1$	=	18
XIX	→	$10 + (10 - 1)$	=	19
LXXII	→	$50 + 10 + 10 + 1 + 1$	=	72
CI	→	$100 + 1$	=	101
MMVII	→	$1000 + 1000 + 5 + 1 + 1$	=	2007
MDC	→	$1000 + 500 + 100$	=	1600

3-1 INTRODUCTION

Data today comes in different forms including numbers, text, audio, image and video (Figure 3.1).

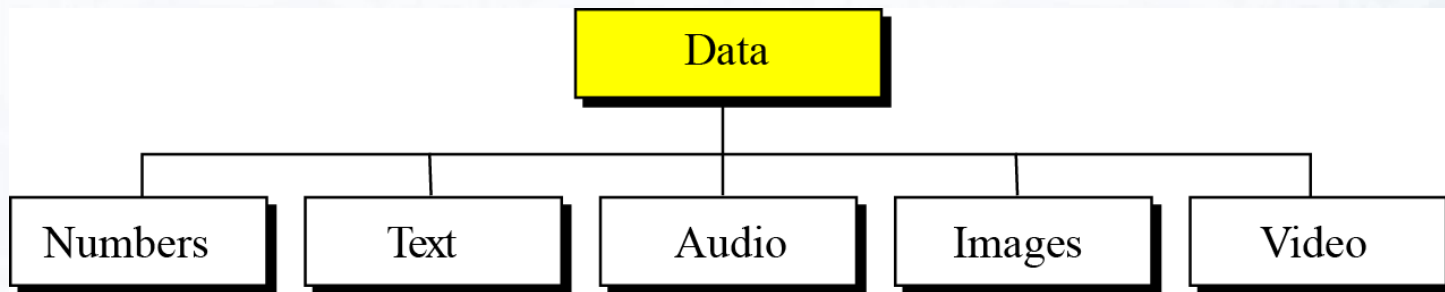


Figure 3.1 Different types of data



The computer industry uses the term “multimedia” to define information that contains numbers, text, images, audio and video.

Data inside the computer

All data types are transformed into a uniform representation when they are stored in a computer and transformed back to their original form when retrieved. This universal representation is called a **bit pattern**.



1 0 0 0 1 0 1 0 1 1 1 1 1 1

Figure 3.2 A bit pattern



wideland
Web Information and Data Engineering Laboratory

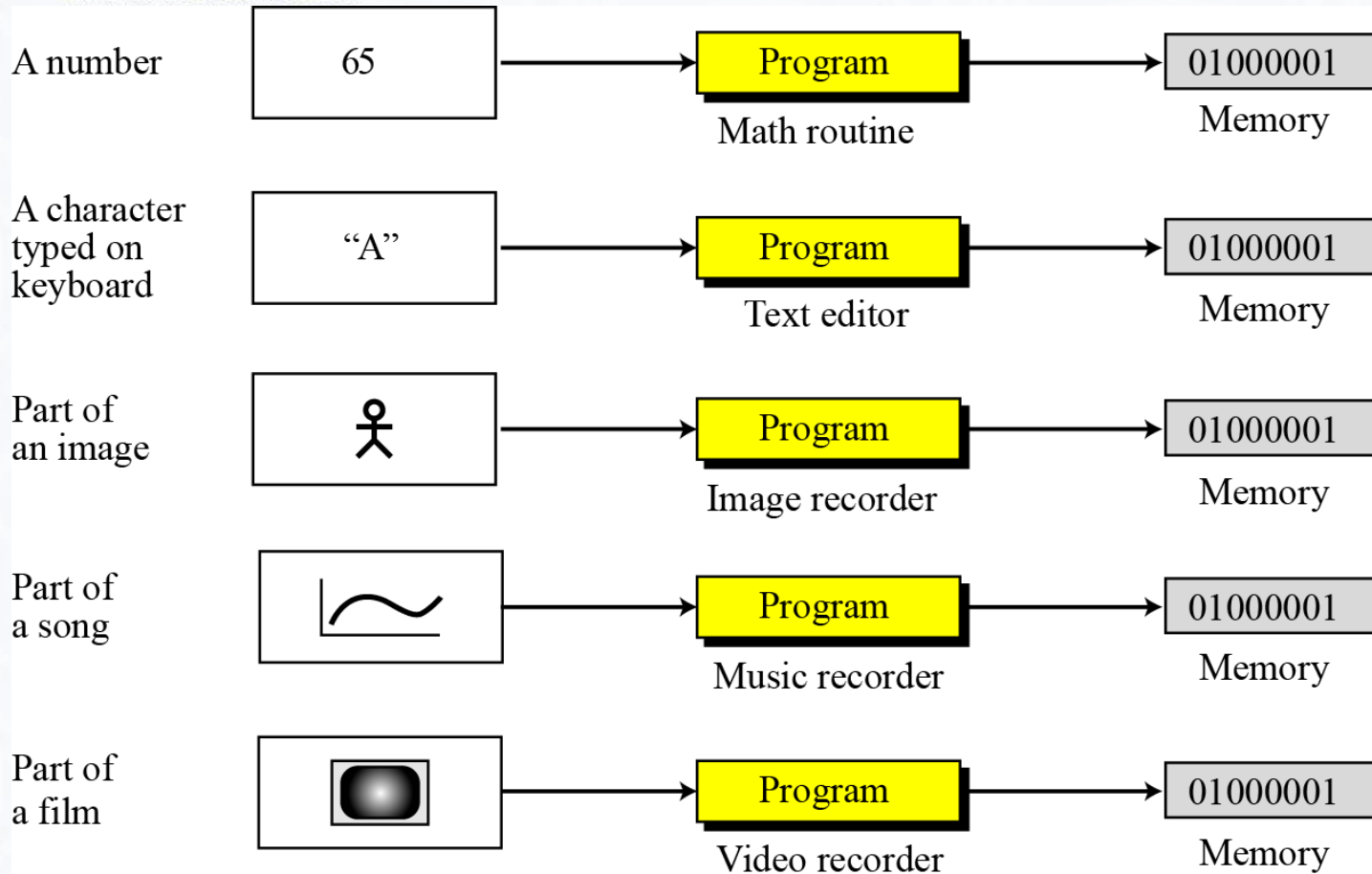


Figure 3.3 Storage of different data types

3-2 STORING NUMBERS

A **number** is converted to the binary system before being stored in the computer memory, as described in Chapter 2. However, there are still two issues that need to be handled:

1. How to store the **sign**(+,- 正負號) of the number.
2. How to show the **decimal point**(小數點).

Unsigned representation

An **unsigned integer** is an integer that can never be negative and can take only 0 or positive values. Its range is between 0 and positive infinity.

An input device stores an unsigned integer using the following steps:

1. The integer is converted to binary.
2. If the number of bits is **less than n** , **0s are added** to the left.

Example 3.1

Store 7 in an 8-bit memory location using unsigned representation.

Solution

First convert the integer to binary, $(111)_2$. Add five 0s to make a total of eight bits, $(00000111)_2$. The integer is stored in the memory location. Note that the subscript 2 is used to emphasize that the integer is binary, but the subscript is not stored in the computer.

Change 7 to binary $\rightarrow$ 1 1 1

Add five bits at the left $\rightarrow$ 0 0 0 0 0 1 1 1

Example 3.2

Store 258 in a 16-bit memory location.

Solution

First convert the integer to binary $(100000010)_2$. Add seven 0s to make a total of sixteen bits, $(0000000100000010)_2$. The integer is stored in the memory location.

Change 258 to binary	→	1	0	0	0	0	0	0	1	0						
Add seven bits at the left	→	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0

n digits 可以放 b base 的最大整數 $= b^n - 1$

Figure 3.5 shows what happens if we try to store an integer that is larger than $2^4 - 1 = 15$ in a memory location that can only hold four bits.

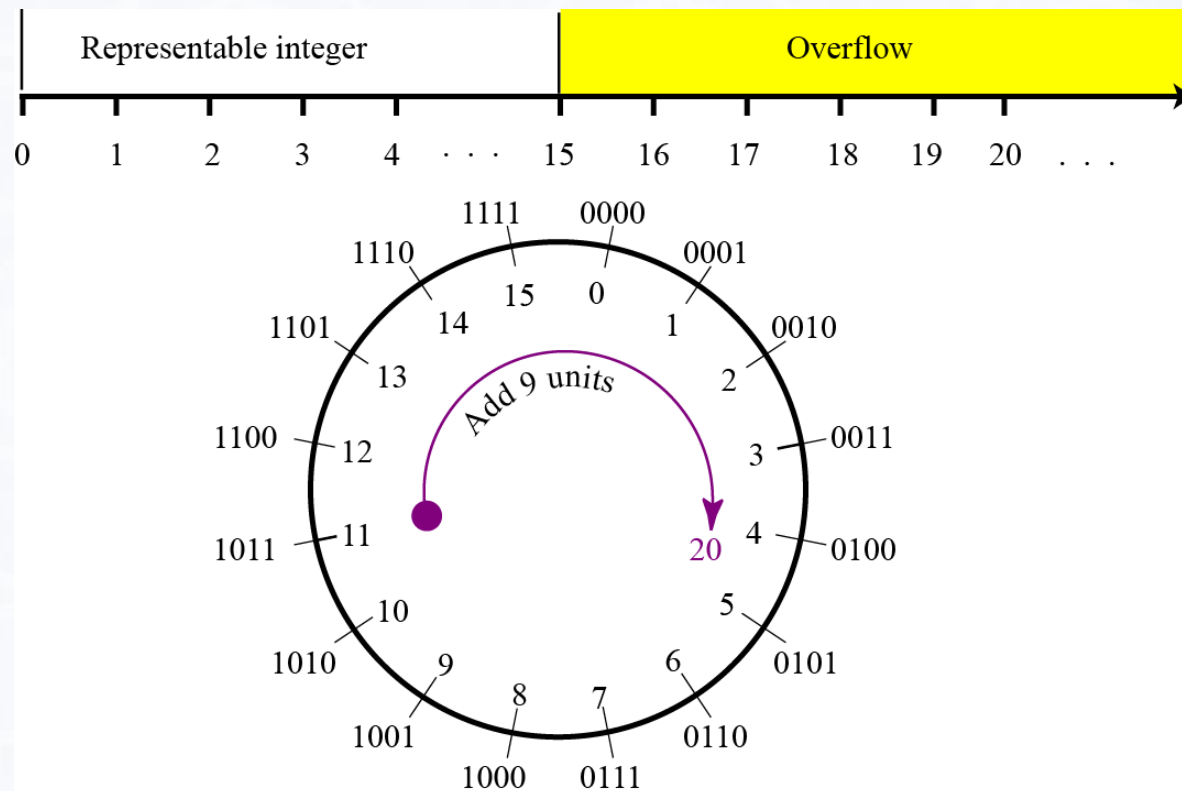


Figure 3.5 Overflow in unsigned integers

Sign-and-magnitude representation

In this method, the available range for unsigned integers (0 to $2^n - 1$) is divided into two equal sub-ranges. The first half represents positive integers, the second half, negative integers.

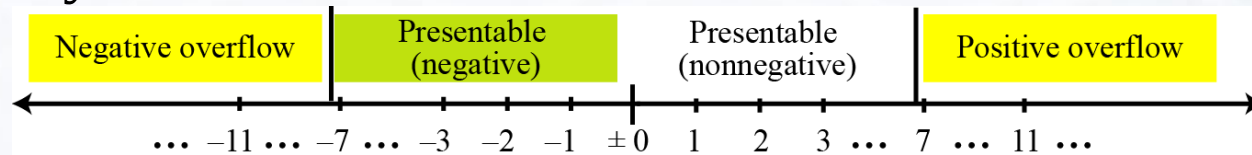
0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0	1	2	3	4	5	6	7	-0	-1	-2	-3	-4	-5	-6	-7

Figure 3.6 Sign-and-magnitude representation

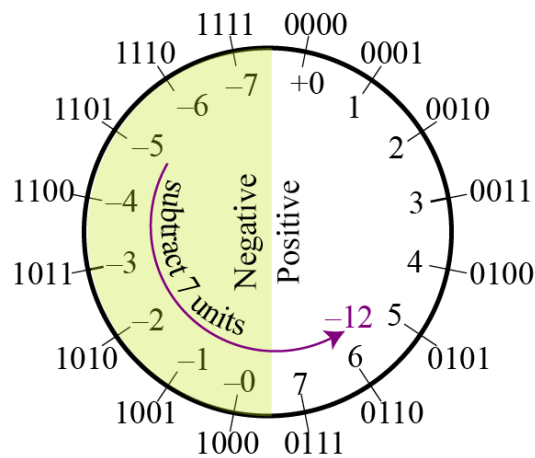


In sign-and-magnitude representation, the leftmost bit defines the sign of the integer. If it is **0**, the integer is positive. If it is **1**, the integer is negative.

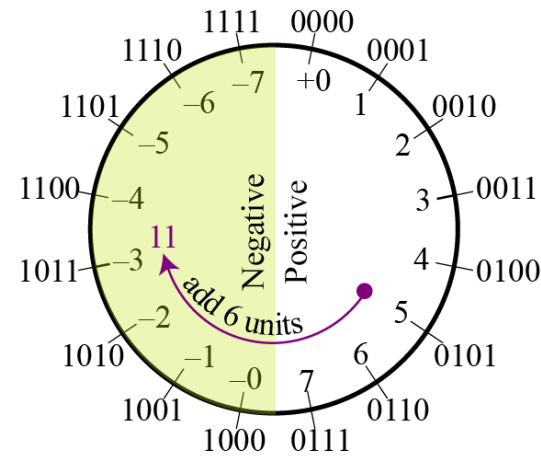
Figure 3.7 shows both positive and negative overflow when storing an integer in sign-and-magnitude representation using a 4-bit memory location.



a. Linear characteristic of an integer in sign-and-magnitude format



b. Negative overflow



c. Positive overflow

Figure 3.7 Overflow in sign-and-magnitude representation

Two's complement representation

Almost all computers use two's complement representation to store a signed integer in an n -bit memory location. In this method, the available range for an unsigned integer of $(0 \text{ to } 2^n - 1)$ is divided into two equal sub-ranges. The first sub-range is used to represent nonnegative integers, the second half to represent negative integers. The bit patterns are then assigned to negative and nonnegative (zero and positive) integers as shown in Figure 3.8.

Complements

❖ Radix complement

- The r 's complement of an n -digit number N in base r is defined as $r^n - N$, for $N \neq 0$ and 0 for $N = 0$
 - The 10's complement of 012398 is 987602
 - The 10's complement of 246700 is 753300
 - The 2's complement of 1101100 is 0010100
 - The 2's complement of 0110111 is 1001001

$$\begin{array}{r} 1000000 \\ - 012398 \\ \hline 987602 \end{array}$$

Note:

1. If the original number N contains a radix point?
2. The complement of the complement?

Complements

❖ Diminished radix complement

- Given a number N in base r having n digits, the $(r-1)$'s complement of N is defined as $(r^n - 1) - N$
 - The 9's complement of 546700 is $999999 - 546700 = 453299$
 - The 9's complement of 012398 is $999999 - 012398 = 987601$
 - The 1's complement of 1011000 is 0100111
 - The 1's complement of 0101101 is 1010010

r' 's complement

- ❖ r' 's complement = $r-1'$'s complement + 1
- ❖ $2'$'s complement = $1'$'s complement + 1

One's Complementing

Before we discuss this representation further, we need to introduce two operations. The first is called **one's complementing** or taking the one's complement of an integer. The operation can be applied to any integer, positive or negative. This operation simply reverses (flips) each bit. A 0-bit is changed to a 1-bit, a 1-bit is changed to a 0-bit.

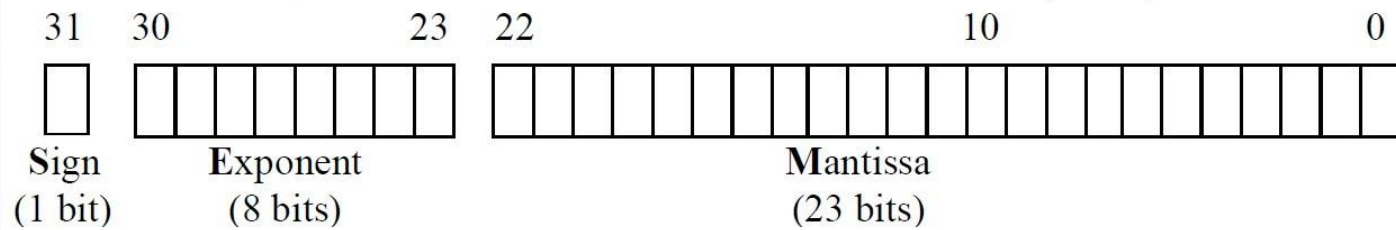
Example 3.8

The following shows how we take the one's complement of the integer 00110110.

Original pattern	0	0	1	1	0	1	1	0
After applying one's complement operation	1	1	0	0	1	0	0	1

In modern computer

- ❖ 2's complement as a negative value.
- ❖ Floating point for non-integer.
 - EXCESS 127 or EXCESS 1023



S	E	M
1	10010100	00000000111000100001111

$$(1.00000000111000100001111)_2 \times 2^{21} = -2,104,378.75$$

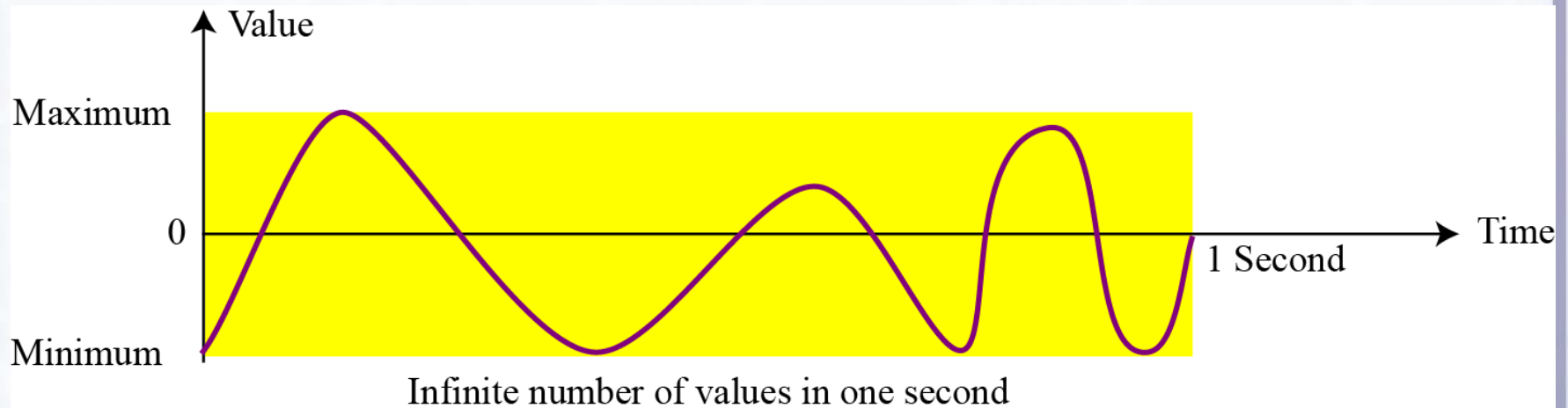


Figure 3.15 An audio signal

Sampling

If we cannot record all the values of a an audio signal over an interval, we can record some of them. Sampling means that we select only a finite number of points on the analog signal, measure their values, and record them.

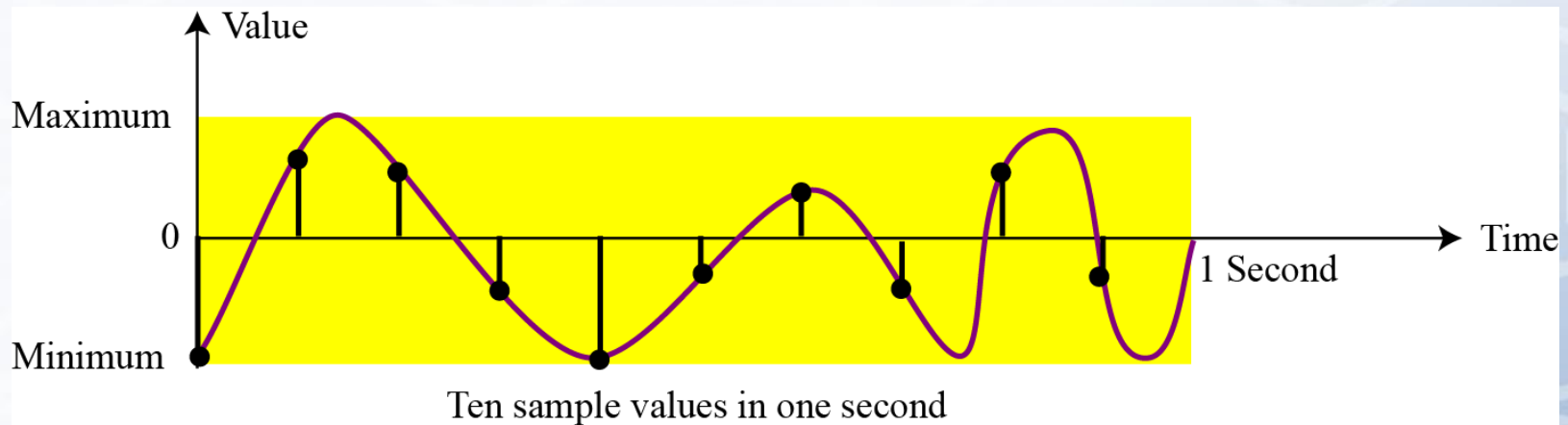


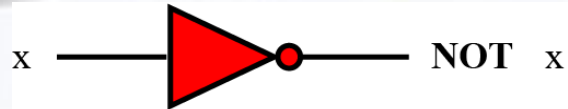
Figure 3.16 Sampling an audio signal

3-5 STORING IMAGES

Images are stored in computers using two different techniques: **raster graphics** and **vector graphics**.

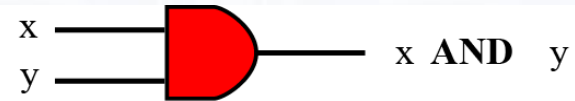
Raster graphics

Raster graphics (or **bitmap graphics**) is used when we need to store an analog image such as a photograph. A photograph consists of analog data, similar to audio information. The difference is that the intensity (color) of data varies in space instead of in time. This means that data must be sampled. However, sampling in this case is normally called **scanning**. The samples are called **pixels** (picture elements).



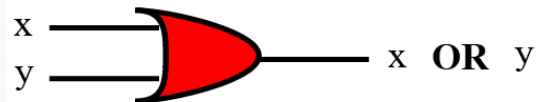
NOT

x	NOT x
0	1
1	0



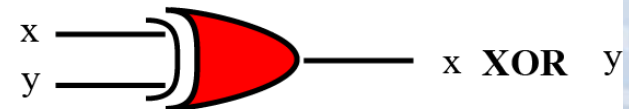
AND

x	y	x AND y
0	0	0
0	1	0
1	0	0
1	1	1



OR

x	y	x OR y
0	0	0
0	1	1
1	0	1
1	1	1



XOR

x	y	x XOR y
0	0	0
0	1	1
1	0	1
1	1	0

Figure 4.1 Logic operations at the bit level

Logic operations at pattern level

The same four operators (NOT, AND, OR, and XOR) can be applied to an n -bit pattern. The effect is the same as applying each operator to each individual bit for NOT and to each corresponding pair of bits for the other three operators. Figure 4.2 shows these four operators with input and output patterns.

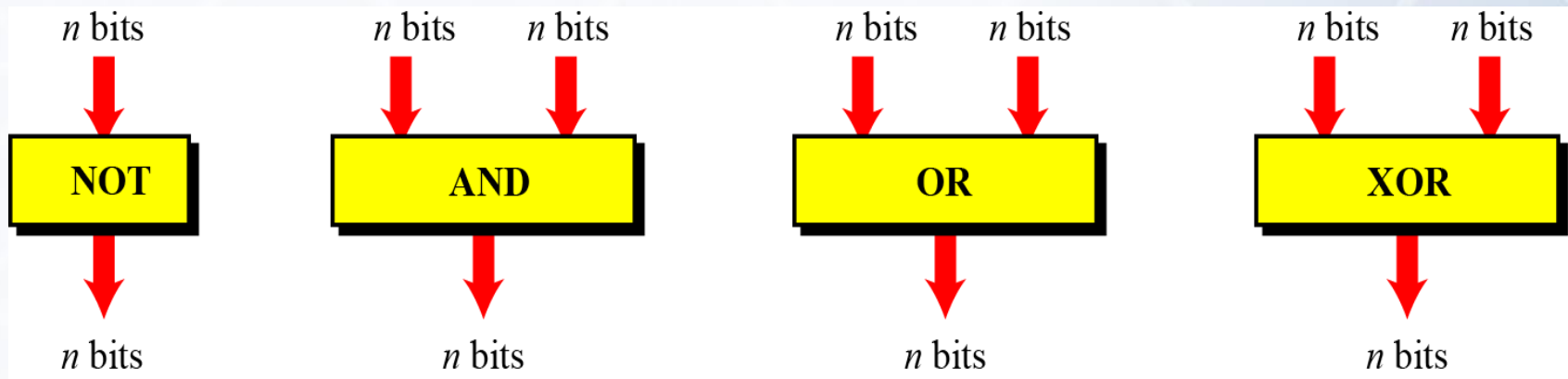


Figure 4.2 Logic operators applied to bit patterns

Example 4.3

Use the NOT operator on the bit pattern 10011000.

Solution

The solution is shown below. Note that the NOT operator changes every 0 to 1 and every 1 to 0.

NOT	1	0	0	1	1	0	0	0	Input
	0	1	1	0	0	1	1	1	Output

Example 4.4

Use the AND operator on the bit patterns 10011000 and 00101010.

Solution

The solution is shown below. Note that only one bit in the output is 1, where both corresponding inputs are 1s.

	1	0	0	1	1	0	0	0	Input 1
AND	0	0	1	0	1	0	1	0	Input 2
	0	0	0	0	1	0	0	0	Output

Applications

The four logic operations can be used to modify a bit pattern.

- ☐ **Complementing (NOT)**
- ☐ **Unsetting (AND)**
- ☐ **Setting (OR)**
- ☐ **Flipping (XOR)**

Example 4.7

Use a mask to unset (clear) the five leftmost bits of a pattern. Test the mask with the pattern 10100110.

Solution

The mask is 00000111. The result of applying the mask is:

	1	0	1	0	0	1	1	0	Input
AND	0	0	0	0	0	1	1	1	Mask
	0	0	0	0	0	1	1	0	Output

[illegible]

Example 4.9

Use a mask to flip the five leftmost bits of a pattern. Test the mask with the pattern 10100110.

Solution

The mask is 11111000. The result of applying the mask is:

	1	0	1	0	0	1	1	0	Input 1
XOR	1	1	1	1	1	0	0	0	Mask
	0	1	0	1	1	1	1	0	Output

4-2 SHIFT OPERATIONS

Shift operations move the bits in a pattern, changing the positions of the bits. They can move bits to the left or to the right. We can divide shift operations into two categories: **logical shift operations** and **arithmetic shift operations**.

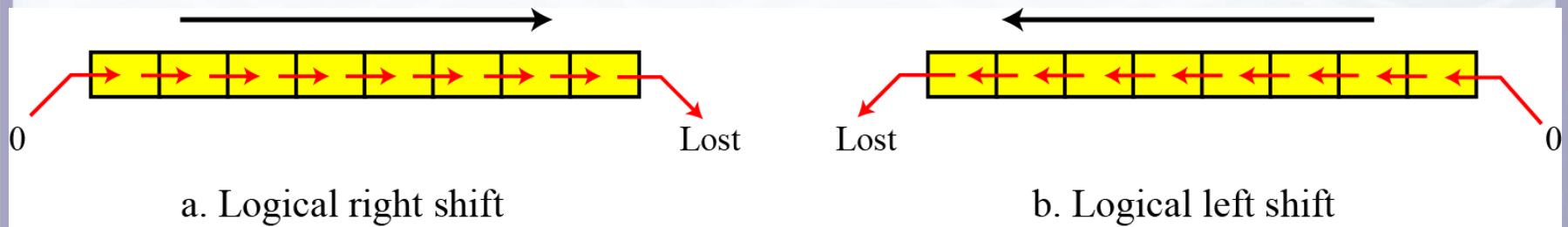


Figure 4.3 Logical shift operations



widelab
Web Information and Data Engineering Laboratory

Two's complement integers

When the subtraction operation is encountered, the computer simply changes it to an addition operation, but makes two's complement of the second number. In other words:

$$\mathbf{A - B \leftrightarrow A + (\overline{B} + 1)}$$

Where $\overline{B}$ is the one's complement of B and $(\overline{B} + 1)$ means the two's complement of B

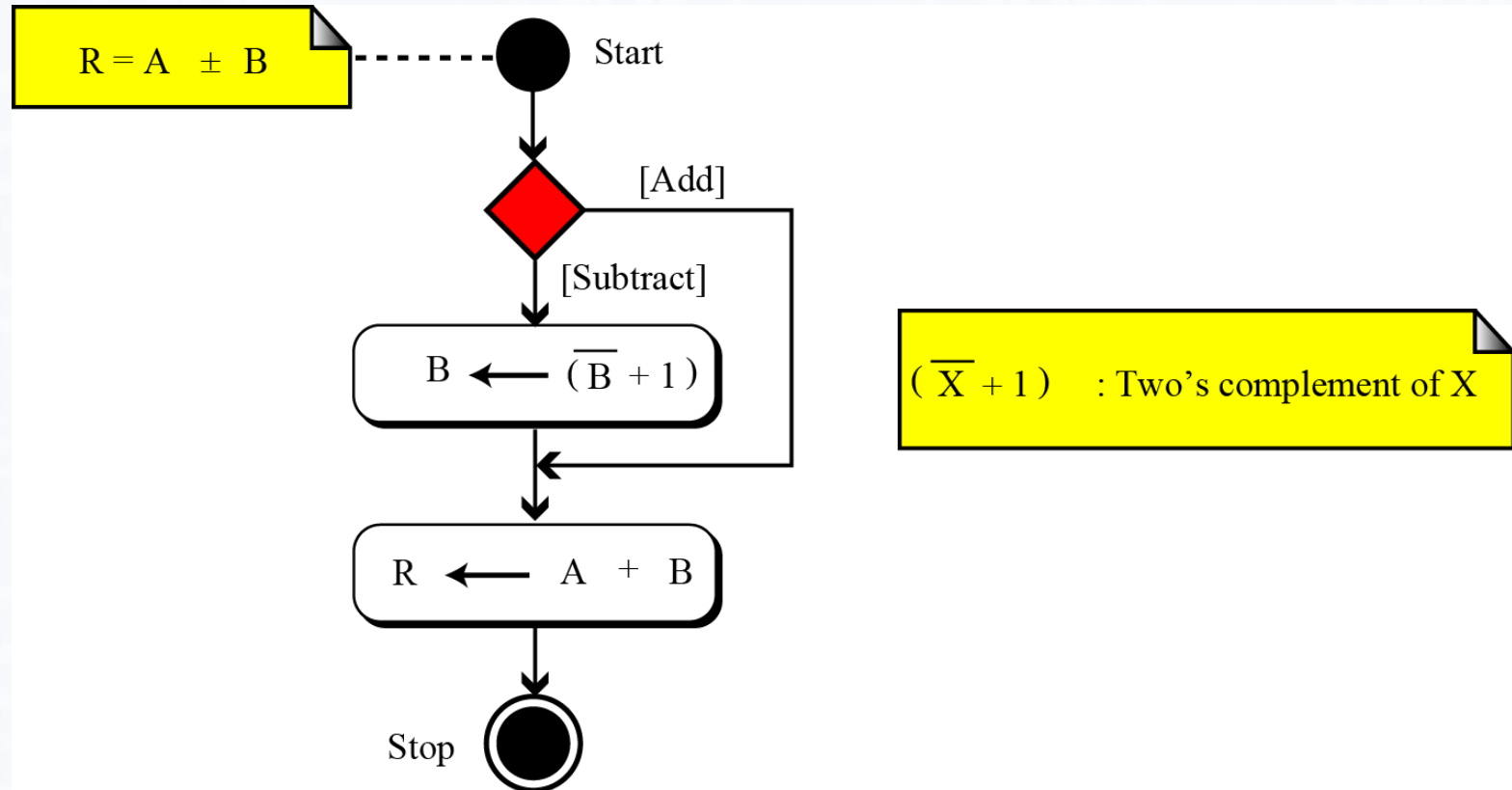


Figure 4.6 Addition and subtraction of integers in two's complement format

We can divide the parts that make up a computer into three broad categories or subsystem: the **central processing unit (CPU)**, the **main memory** and the **input/output subsystem**.

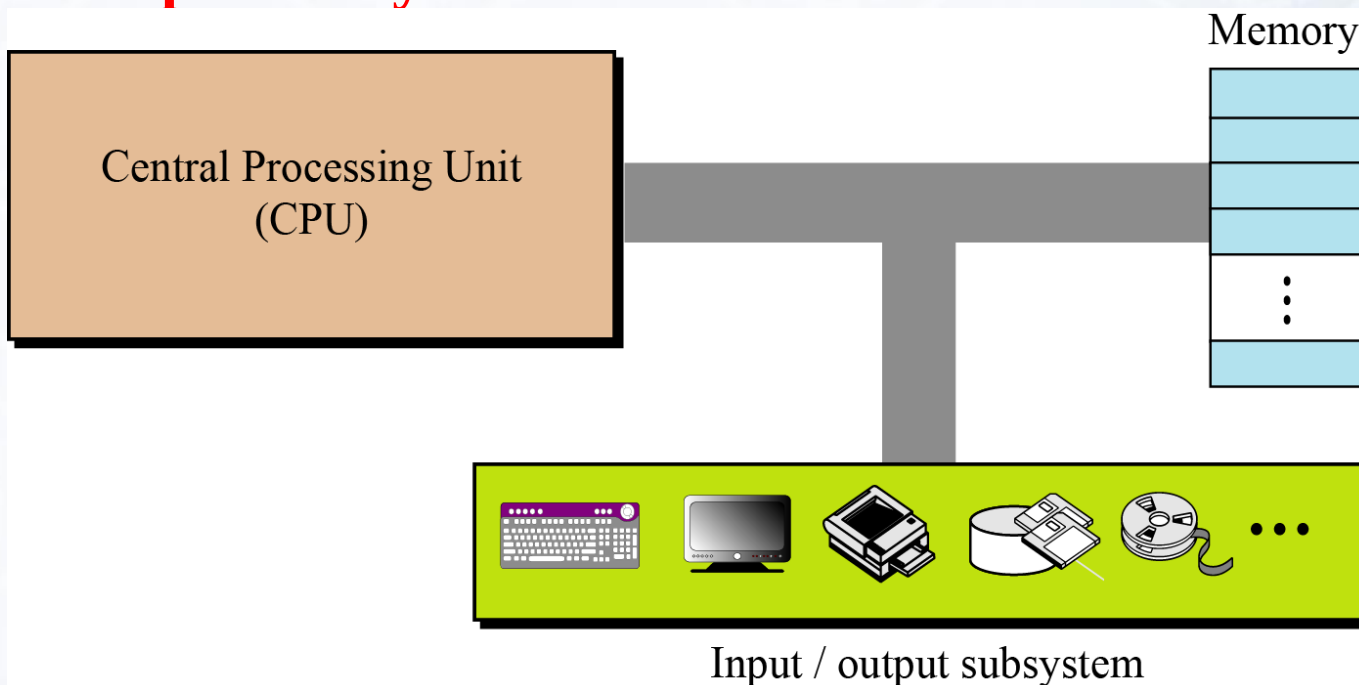


Figure 5.1 Computer hardware (subsystems)



widelab
Web Information and Data Engineering Laboratory

5-1 CENTRAL PROCESSING UNIT

The **central processing unit (CPU)** performs operations on data. In most architectures it has three parts: an **arithmetic logic unit (ALU)**, a **control unit** and a set of **registers**, fast storage locations.

The arithmetic logic unit (ALU)

The central processing unit (CPU) performs operations on data. In most architectures it has three parts: an arithmetic logic unit (ALU), a control unit and a set of registers, fast storage locations (Figure 5.2).

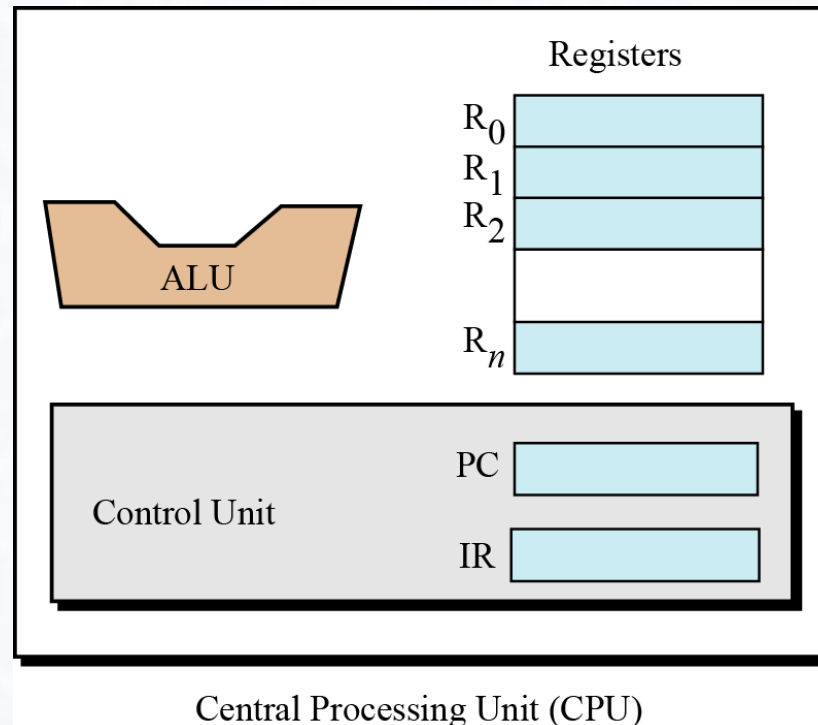


Figure 5.2 Central processing unit (CPU)

Registers

Registers are fast stand-alone storage locations that hold data temporarily. Multiple registers are needed to facilitate the operation of the CPU. Some of these registers are shown in Figure 5.2.

- ☐ Data registers
- ☐ Instruction register
- ☐ Program counter

The control unit

The third part of any CPU is the control unit. The control unit controls the operation of each subsystem. Controlling is achieved through signals sent from the control unit to other subsystems.



widelab
Web Information and Data Engineering Laboratory

Table 5.1 Memory units

<i>Unit</i>	<i>Exact Number of Bytes</i>	<i>Approximation</i>
kilobyte	2^{10} (1024) bytes	10^3 bytes
megabyte	2^{20} (1,048,576) bytes	10^6 bytes
gigabyte	2^{30} (1,073,741,824) bytes	10^9 bytes
terabyte	2^{40} bytes	10^{12} bytes



Memory addresses are defined using unsigned binary integers.

然後勒？

- ❖ Petabytes
- ❖ Exabytes
- ❖ Zettabytes
- ❖ Yottabytes
- ❖ ... 應該沒了吧！

5-3 INPUT/OUTPUT SUBSYSTEM

The third major subsystem in a computer is the collection of devices referred to as the input/output (I/O) subsystem. This subsystem allows a computer to communicate with the outside world and to store programs and data even when the power is off. Input/output devices can be divided into two broad categories: **non-storage** and **storage** devices.

Non-storage devices

Non-storage devices allow the CPU/memory to communicate with the outside world, but they cannot store information.

- ☐ Keyboard and monitor
- ☐ Printer



widelab
Web Information and Data Engineering Laboratory

5-5 PROGRAM EXECUTION

Today, **general-purpose computers** use a set of instructions called a **program** to process data. A computer executes the program to create output data from input data. Both the program and the data are stored in memory.



At the end of this chapter we give some examples of how a hypothetical simple computer executes a program.

Machine cycle

The CPU uses repeating **machine cycles** to execute instructions in the program, one by one, from beginning to end. A simplified cycle can consist of three phases: **fetch**, **decode** and **execute** (Figure 5.19).

Fetch : copy the next instruction into the instruction register in the CPU

Decode : control unit will decode the instruction into the binary code

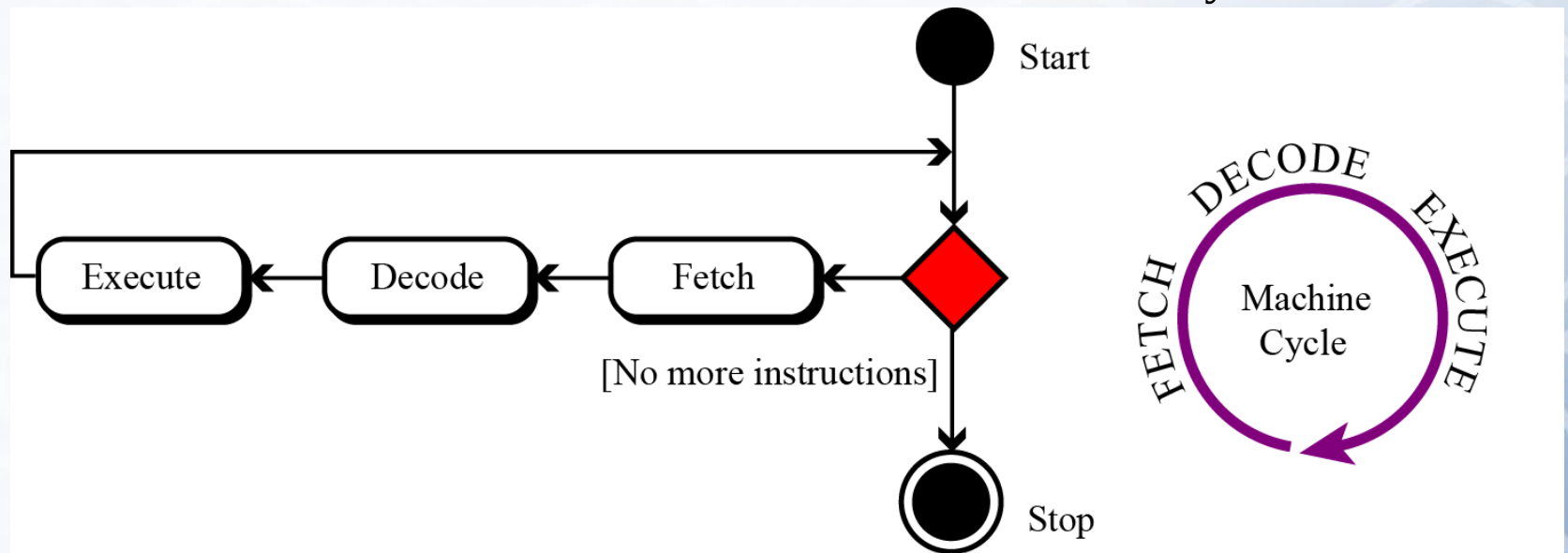


Figure 5.19 The steps of a cycle

5-7 A SIMPLE COMPUTER

To explain the architecture of computers as well as their instruction processing, we introduce a simple (unrealistic) computer, as shown in Figure 5.30. Our simple computer has three components: CPU, memory and an input/output subsystem.



WIDE LAB

Hsien-Tsung Chang@WIDE LAB

Department of Computer Science and Information Engineering
Chang Gung University

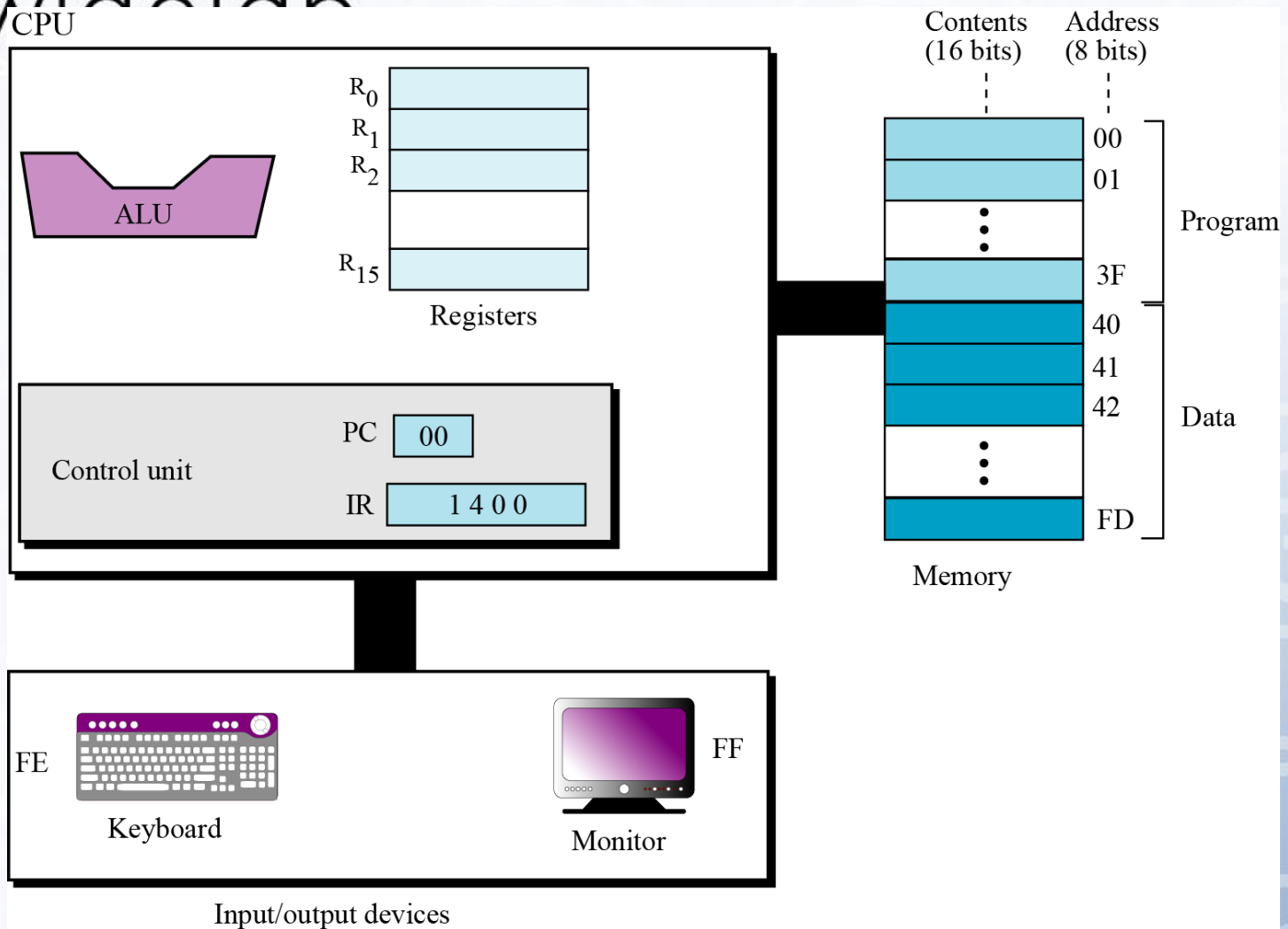
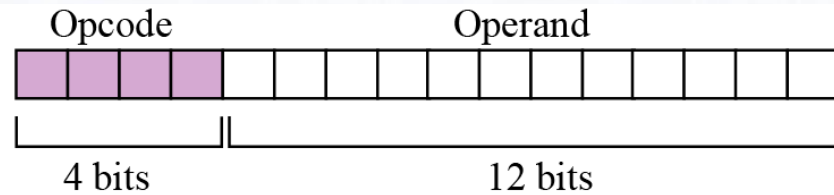


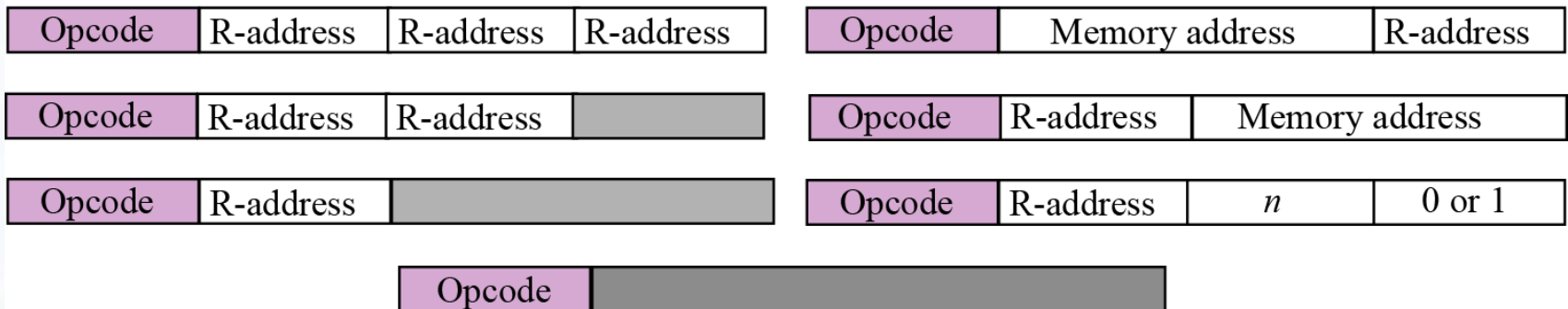
Figure 5.30 The components of a simple computer

Instruction set

Our simple computer is capable of having a set of sixteen instructions, although we are using only fourteen of these instructions. Each computer instruction consists of two parts: the **operation code (opcode)** and the **operand (s)**. The opcode specifies the type of operation to be performed on the operand (s). Each instruction consists of sixteen bits divided into four 4-bit fields. The leftmost field contains the opcode and the other three fields contains the operand or address of operand (s), as shown in Figure 5.31.



a. Instruction format



b. Instruction types

Figure 5.31 Format and different instruction types

Processing the instructions

Our simple computer, like most computers, uses machine cycles. A cycle is made of three phases: fetch, decode and execute. During the fetch phase, the instruction whose address is determined by the PC is obtained from the memory and loaded into the IR. The PC is then incremented to point to the next instruction. During the decode phase, the instruction in IR is decoded and the required operands are fetched from the register or from memory. During the execute phase, the instruction is executed and the results are placed in the appropriate memory location or the register. Once the third phase is completed, the control unit starts the cycle again, but now the PC is pointing to the next instruction. The process continues until the CPU reaches a HALT instruction.

Table 5.4 List of instructions for the simple computer

Instruction	Code Operands				Action
	d ₁	d ₂	d ₃	d ₄	
HALT	0				Stops the execution of the program
LOAD	1	R _D	M _S		$R_D \leftarrow M_S$
STORE	2	M _D		R _S	$M_D \leftarrow R_S$
ADDI	3	R _D	R _{S1}	R _{S2}	$R_D \leftarrow R_{S1} + R_{S2}$
ADDF	4	R _D	R _{S1}	R _{S2}	$R_D \leftarrow R_{S1} + R_{S2}$
MOVE	5	R _D	R _S		$R_D \leftarrow R_S$
NOT	6	R _D	R _S		$R_D \leftarrow \overline{R_S}$
AND	7	R _D	R _{S1}	R _{S2}	$R_D \leftarrow R_{S1} \text{ AND } R_{S2}$
OR	8	R _D	R _{S1}	R _{S2}	$R_D \leftarrow R_{S1} \text{ OR } R_{S2}$
XOR	9	R _D	R _{S1}	R _{S2}	$R_D \leftarrow R_{S1} \text{ XOR } R_{S2}$
INC	A	R			$R \leftarrow R + 1$
DEC	B	R			$R \leftarrow R - 1$
ROTATE	C	R	n	0 or 1	$\text{Rot}_n R$
JUMP	D	R	n		IF $R_0 \neq R$ then $PC = n$, otherwise continue

Key: R_S, R_{S1}, R_{S2}: Hexadecimal address of source registers
 R_D: Hexadecimal address of destination register
 M_S: Hexadecimal address of source memory location
 M_D: Hexadecimal address of destination memory location
 n: hexadecimal number
 d₁, d₂, d₃, d₄: First, second, third, and fourth hexadecimal digits

An example

Let us show how our simple computer can add two integers A and B and create the result as C . We assume that integers are in two's complement format. Mathematically, we show this operation as:

$$C = A + B$$

We assume that the first two integers are stored in memory locations $(40)_{16}$ and $(41)_{16}$ and the result should be stored in memory location $(42)_{16}$. To do the simple addition needs five instructions, as shown next:

1. Load the contents of M_{40} into register R_0 ($R_0 \leftarrow M_{40}$).
2. Load the contents of M_{41} into register R_1 ($R_1 \leftarrow M_{41}$).
3. Add the contents of R_0 and R_1 and place the result in R_2 ($R_2 \leftarrow R_0 + R_1$).
4. Store the contents R_2 in M_{42} ($M_{42} \leftarrow R_2$).
5. Halt.

In the language of our simple computer, these five instructions are encoded as:

<i>Code</i>	<i>Interpretation</i>			
$(1040)_{16}$	1: LOAD	0: R_0	40: M_{40}	
$(1141)_{16}$	1: LOAD	1: R_1	41: M_{41}	
$(3201)_{16}$	3: ADDI	2: R_2	0: R_0	1: R_1
$(2422)_{16}$	2: STORE	42: M_{42}		2: R_2
$(0000)_{16}$	0: HALT			

Storing program and data

We can store the five-line program in memory starting from location $(00)_{16}$ to $(04)_{16}$. We already know that the data needs to be stored in memory locations $(40)_{16}$, $(41)_{16}$, and $(42)_{16}$.

Cycles

Our computer uses one cycle per instruction. If we have a small program with five instructions, we need five cycles. We also know that each cycle is normally made up of three steps: fetch, decode, execute. Assume for the moment that we need to add $161 + 254 = 415$. The numbers are shown in memory in hexadecimal is, $(00A1)_{16}$, $(00FE)_{16}$, and $(019F)_{16}$.

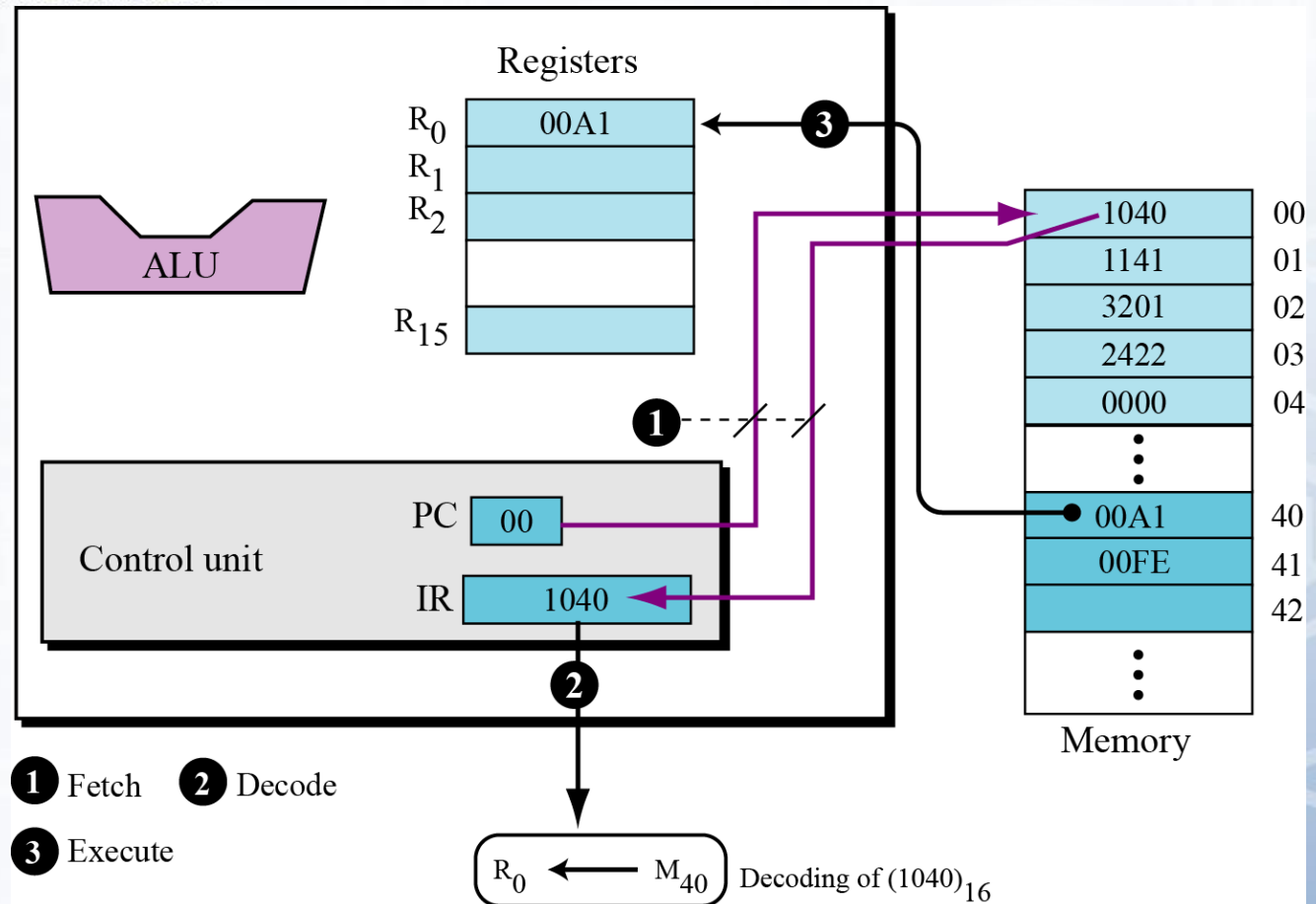


Figure 5.32 Status of cycle 1

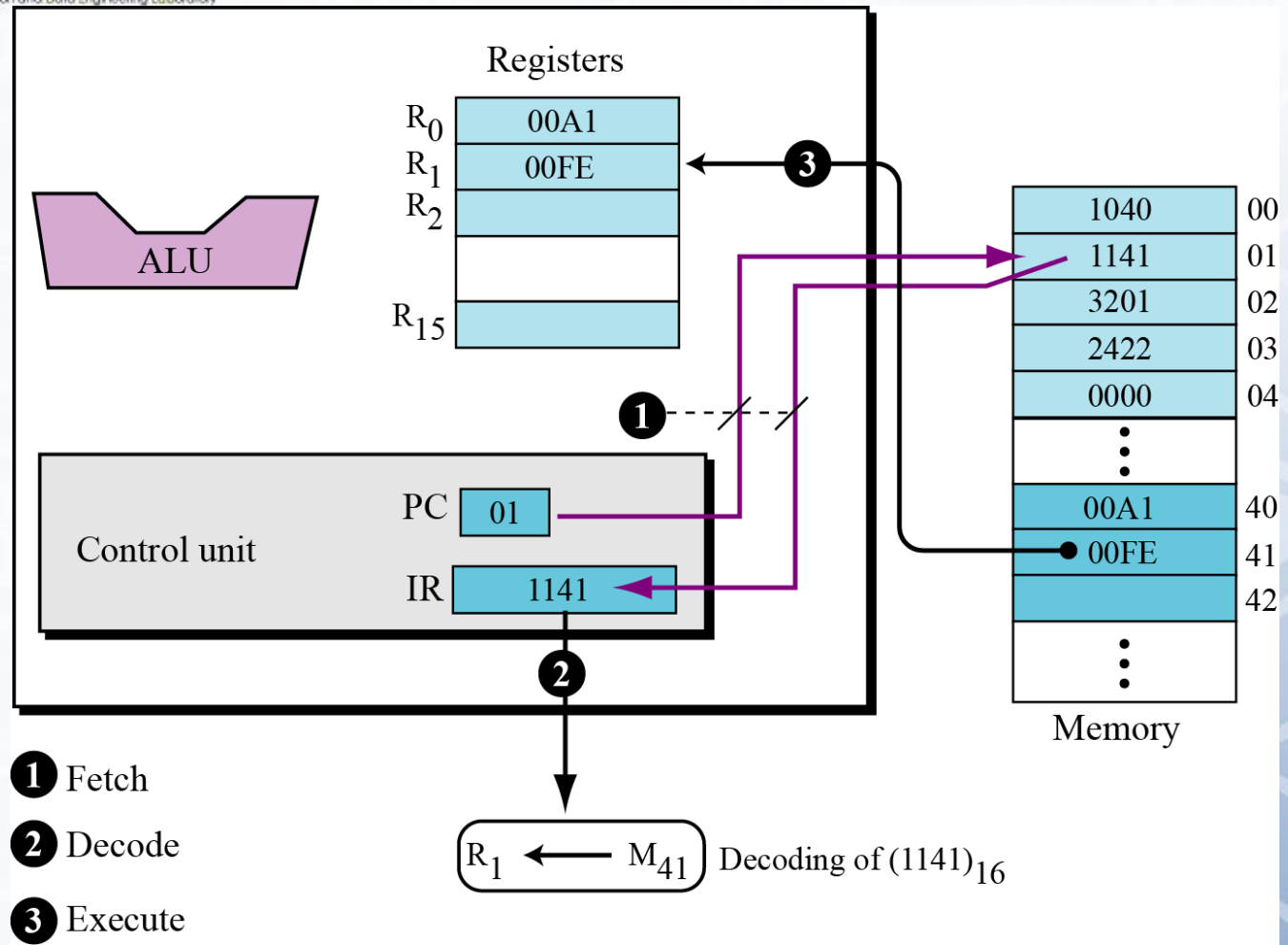


Figure 5.33 Status of cycle 2

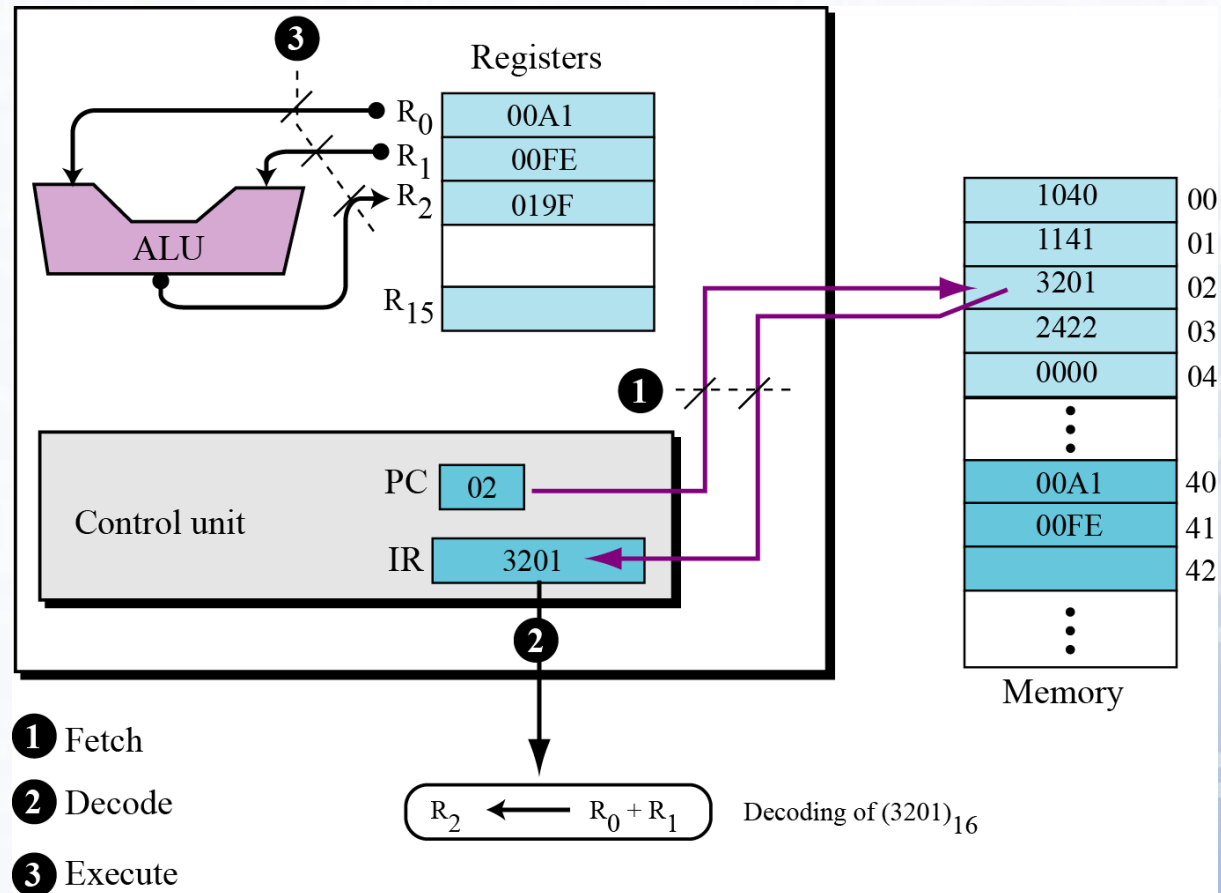


Figure 5.34 Status of cycle 3

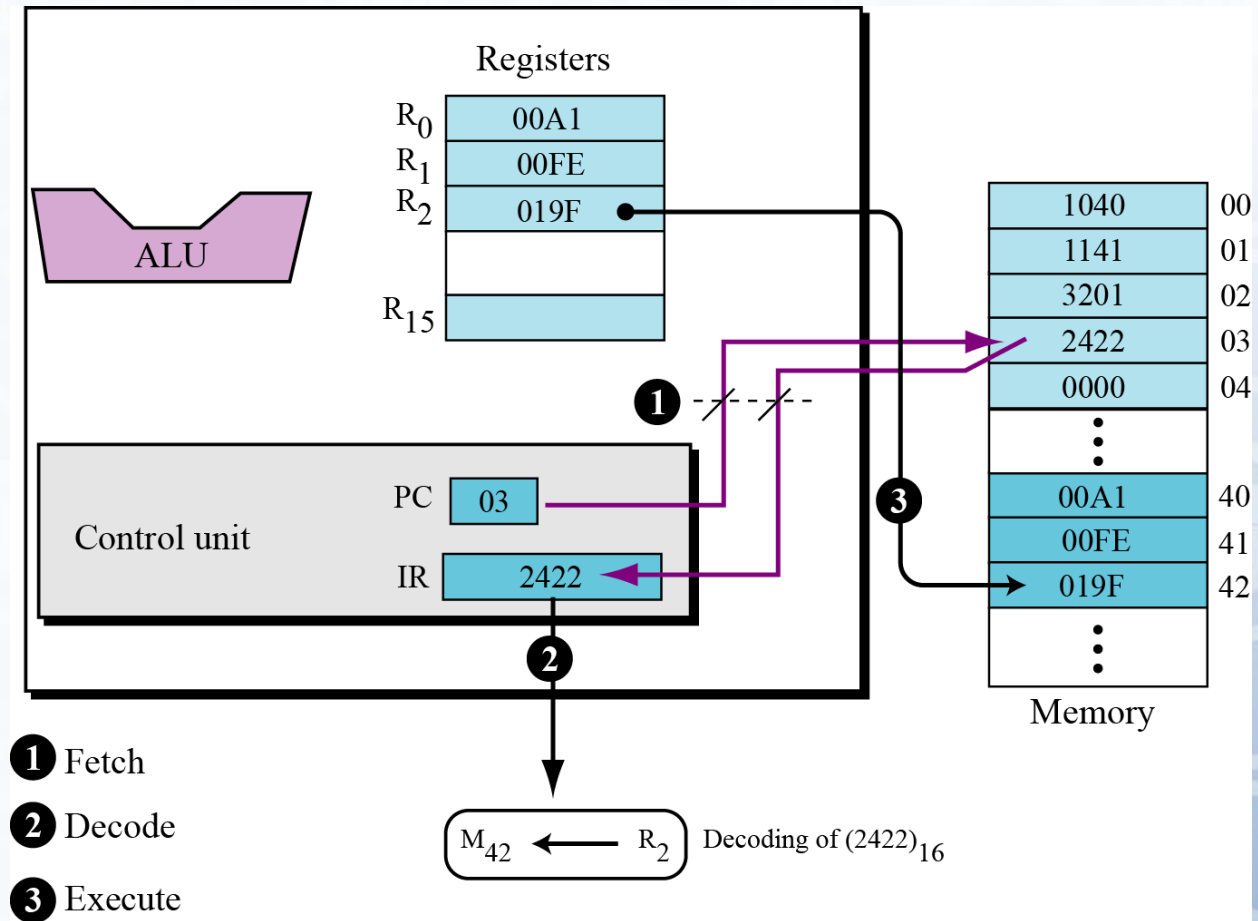


Figure 5.35 Status of cycle 4

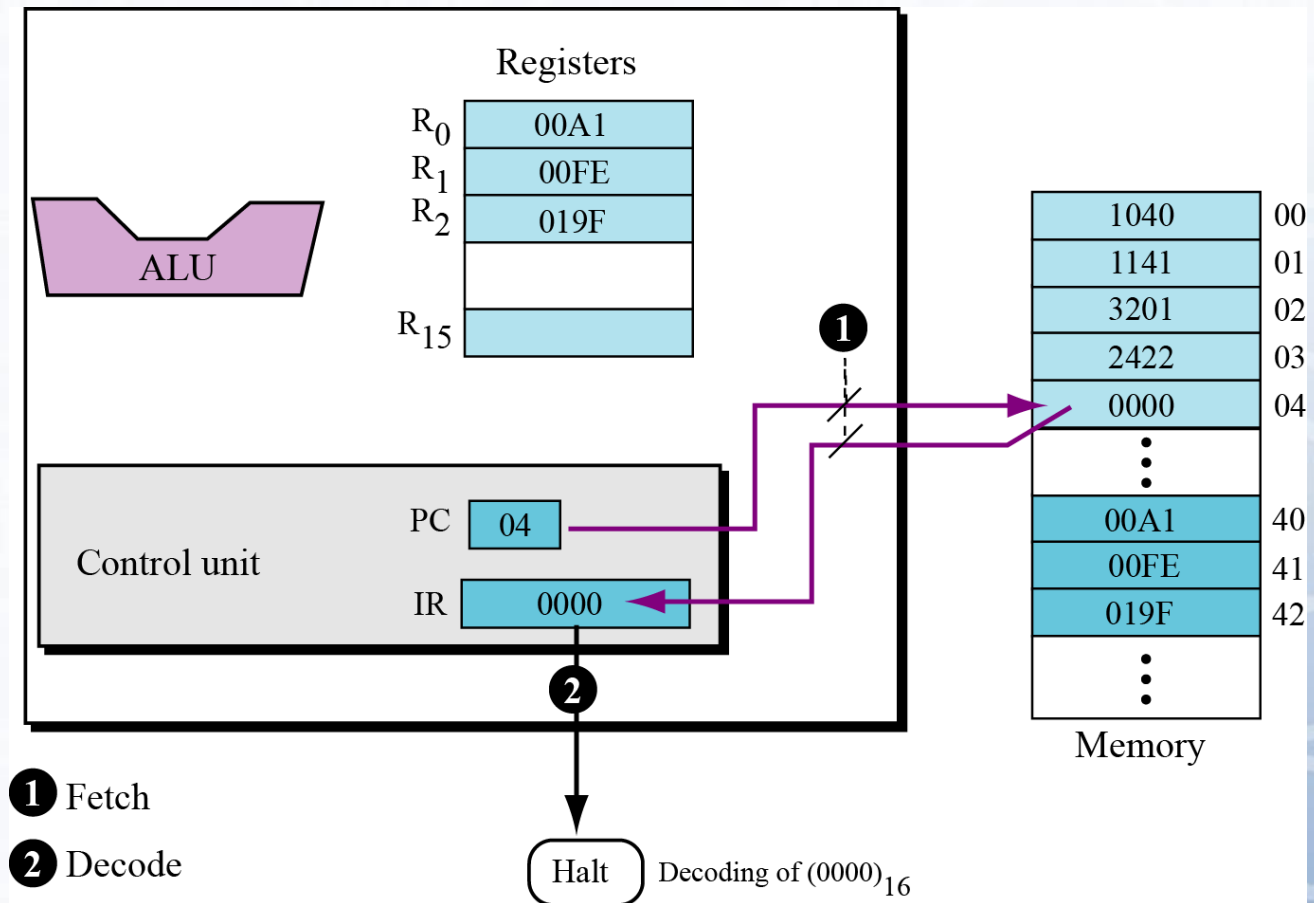


Figure 5.36 Status of cycle 5

Another example

In the previous example we assumed that the two integers to be added were already in memory. We also assume that the result of addition will be held in memory. You may ask how we can store the two integers we want to add in memory, or how we use the result when it is stored in the memory. In a real situation, we enter the first two integers into memory using an input device such as keyboard, and we display the third integer through an output device such as a monitor. Getting data via an input device is normally called a read operation, while sending data to an output device is normally called a write operation. To make our previous program more practical, we need modify it as follows:

1. Read an integer into M_{40} .
2. $R_0 \leftarrow M_{40}$.
3. Read an integer into M_{41} .
4. $R_1 \leftarrow M_{41}$.
5. $R_2 \leftarrow R_0 + R_1$.
6. $M_{42} \leftarrow R_2$.
7. Write the integer from M_{42} .
8. Halt.

In our computer we can simulate read and write operations using the LOAD and STORE instruction. Furthermore, LOAD and STORE read data input to the CPU and write data from the CPU. We need two instructions to read data into memory or write data out of memory. The read operation is: